

# Микросервисы: вопросы с собеседований

## О нас

FAANG School – IT онлайн-школа с первоклассными образовательными продуктами, которая выпускает разработчиков с гибким мышлением и мощным стеком.

## Наша миссия

Помогаем реализовать амбиции, стать профессиональными разработчиками и получить работу мечты в лучших IT-компаниях в кратчайшие сроки!

### Что такое Java Буткемп?

Java Буткемп — это интенсивное обучение в формате командной работы над мощным проектом, где ты получишь максимально приближенный к реальному опыт для успешного трудоустройства с высокой зарплатой в кратчайшие сроки.

Для этого мы адаптировали практики разработки BigTech-компаний, внедрили передовые инструменты индустрии и организовали работу по гибкой методологии Scrum, чтобы ты выделялся среди других кандидатов на рынке.

**Стремишься стать востребованным разработчиком и получить работу мечты в кратчайшие сроки? Скорее записывайся на Java Буткемп!**

[\*\*ЗАПИСАТЬСЯ НА ЭКСКУРСИЮ\*\*](#)

**Микросервисы** — популярный способ структурирования программных систем. По мере того как компании растут и используют больше облачных хранилищ, микросервисы помогают справляться с увеличивающейся нагрузкой.

В этой шпаргалке мы рассмотрим некоторые ключевые концепции микросервисов и распространенные вопросы, которые возникают на собеседованиях:

- 1. Что такое микросервисы?**
- 2. Какие различия между SOA и микросервисами?**
- 3. Какие проблемы решают микросервисы?**
- 4. Какие проблемы создают микросервисы?**
- 5. Какие основные компоненты архитектуры микросервисов?**
- 6. Какие популярные решения для микросервисов существуют?**
- 7. Что такое реестр сервисов?**
- 8. Что такое API-шлюз?**
- 9. В чем различия между REST и RPC?**
- 10. Что такое менеджер конфигурации?**
- 11. Какие существуют общие подходы к обеспечению отказоустойчивости микросервисов?**
- 12. Как мы управляем распределенными транзакциями?**
- 13. Как выбрать между монолитной и микросервисной архитектурами?**

# 1. Что такое микросервисы?

Тут процитируем Мартина Фаулера и Эдриана Кокрофта по ключевым аспектам микросервисов.



От Мартина Фаулера:



Стиль архитектуры микросервисов — это подход к разработке одного приложения в виде набора небольших сервисов (отдельных программ), каждый из которых работает в своем собственном процессе и общается с помощью легковесных механизмов, часто через HTTP API. Эти сервисы строятся вокруг бизнес-возможностей и могут независимо разворачиваться с помощью полностью автоматизированной системы развертывания (размещения этих сервисов на физических серверах). Центральное управление этими сервисами минимально, и они могут быть написаны на разных языках программирования и использовать различные технологии хранения данных.



И более короткое определение от Эдриана Кокрофта:



«Микросервисы — это свободно связанные сервисы в архитектуре, ориентированной на сервисы, с ограниченными контекстами.»

На нашем [Java Буткемпе](#) мы разбиваем приложение на 9 независимых микросервисов. Каждый микросервис — это небольшое приложение, выполняющее одну конкретную задачу и взаимодействующее с другими через четко определенные интерфейсы. Это позволяет нам разрабатывать, тестировать и масштабировать разные части системы независимо друг от друга. В итоге участники буткемпа создают огромное веб-приложение и работают с каждым микросервисом!

# Ключевые аспекты микросервисов

Можно выделить ключевые аспекты архитектуры микросервисов:

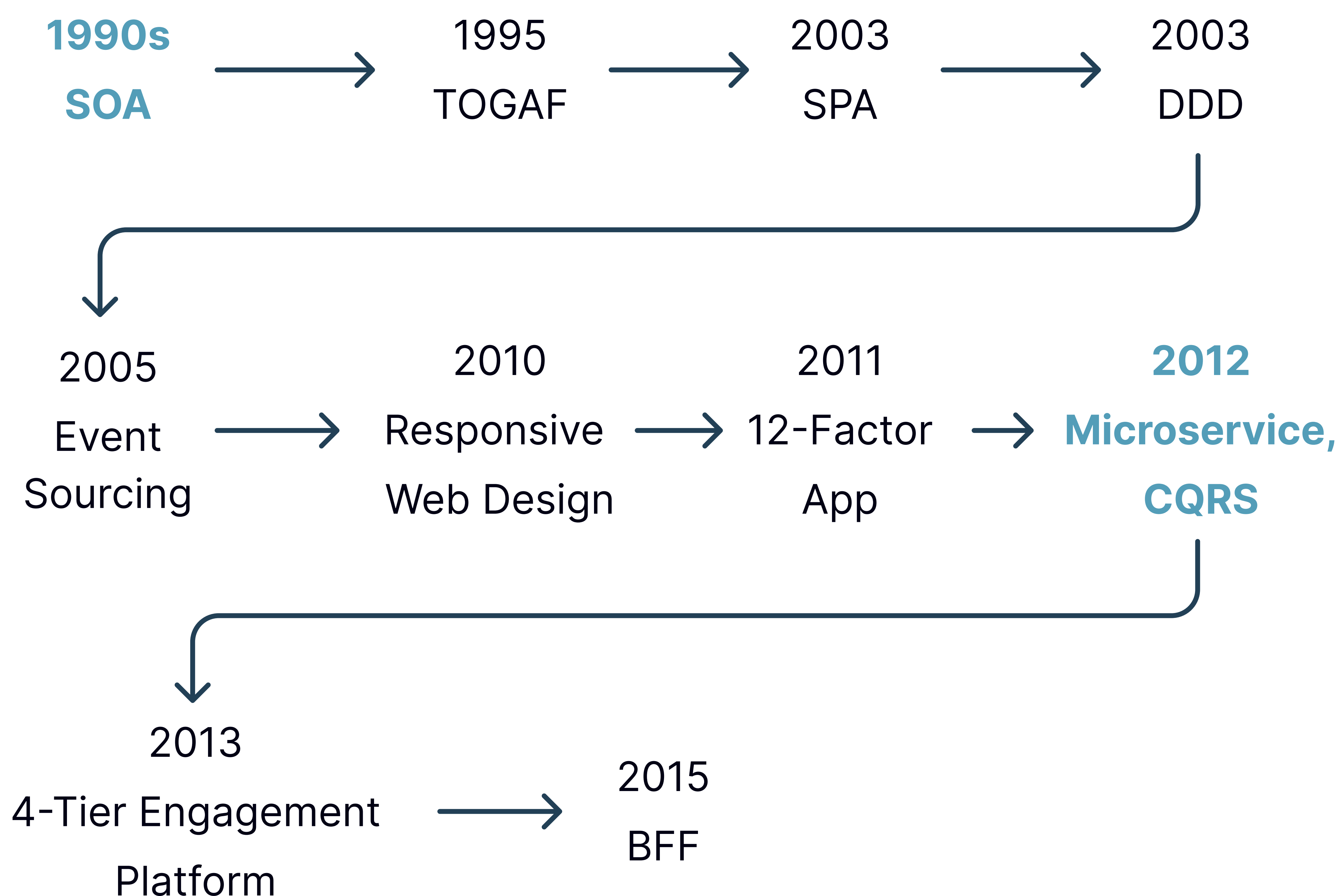
- 1. Разбиение монолитного приложения** на небольшие независимые сервисы (отдельные маленькие приложения). Это позволяет различным продуктовым командам разрабатывать, тестировать и разворачивать сервисы, которые соответствуют конкретным бизнес-возможностям. Полезно для крупных организаций, чтобы разбить монолитные системы и повысить производительность.
- 2. Слабосвязанные сервисы, общающиеся через API.** Компоненты фронтенда/бэкенда общаются через REST, а для межсервисного взаимодействия используется RPC для эффективного запроса/ответа. Это банально быстрее.
- 3. Тщательно спроектированы вокруг ограниченных контекстов.** Каждый сервис имеет четкие границы модулей и инкапсулированную доменную логику, чтобы избежать тесной связи между сервисами. Т.о. каждый микросервис отвечает только за свою часть данных, за свои сущности и только за логику, связанную именно с ними.
- 4. Позволяет эффективно применять практики DevOps.** Это важная часть методологии разработки микросервисов. Небольшие, полнофункциональные команды полностью владеют конкретными сервисами от начала до конца. Контейнеризация, автоматизация и оркестрация контейнеров используются для эффективного развертывания микросервисов.
- 5. Горизонтально масштабируемы по дизайну, устойчивы к сбоям.** Сервисы могут масштабироваться независимо по мере необходимости. Построены с учетом отказоустойчивости. Выросла нагрузка — запустили еще машин с копиями этих микросервисов на них.
- 6. Децентрализованное управление и гибкость.** Команды могут выбирать любую технологию, которая имеет смысл для их сервиса. Один микросервис может быть написан на Java, а другой — на Go, если это необходимо.
- 7. Требуют обширного мониторинга и администрирования из-за:**
  - а. Роста количества сервисов — По мере разбиения монолитов на множество независимых сервисов количество компонентов для отслеживания быстро растет. Все они находятся под разной нагрузкой, какие-то порой падают и т. д. В общем этим «хаосом» нужно управлять.
  - б. Раз уж мы работаем с Docker и Kubernetes, для управления инфраструктурой и развертывания, то мониторинг работы сервиса должен осуществляться на уровне кода приложения с использованием дополнительных инструментов для агрегации логов, метрик и трассировок.

## 2. Какие различия между SOA и микросервисами?

Стиль архитектуры, ориентированной на сервисы (SOA), и архитектура микросервисов являются важными этапами эволюции программной архитектуры. Диаграмма ниже показывает эволюцию ключевых архитектурных стилей.

Архитектура, ориентированная на сервисы, появилась в конце 1990-х годов для помощи в управлении растущей сложностью корпоративных программных систем. В 2000-х годах SOA привлекала больше внимания и получила распространение среди компаний. Однако столкнулась с проблемами сложности реализации.

Затем, в 2010-х годах, архитектура микросервисов появилась в ответ на ограничения SOA. Многие крупные интернет-компании начали внедрять микросервисы для разделения своих сервисов на более мелкие компоненты. Микросервисы набрали обороты с развитием облачных вычислений, поскольку контейнеры и инструменты оркестрации упростили разработку, развертывание и мониторинг микросервисов.

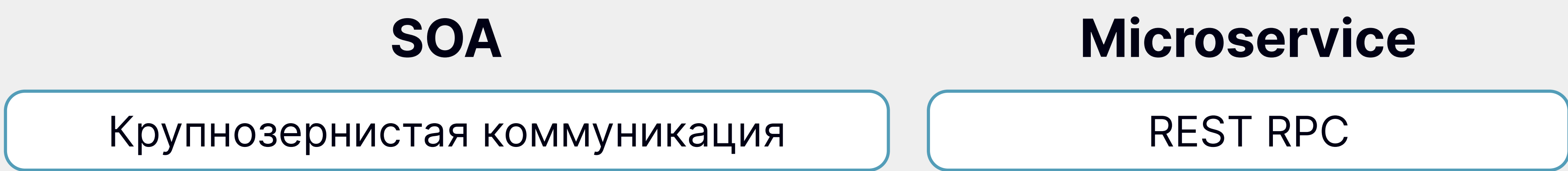
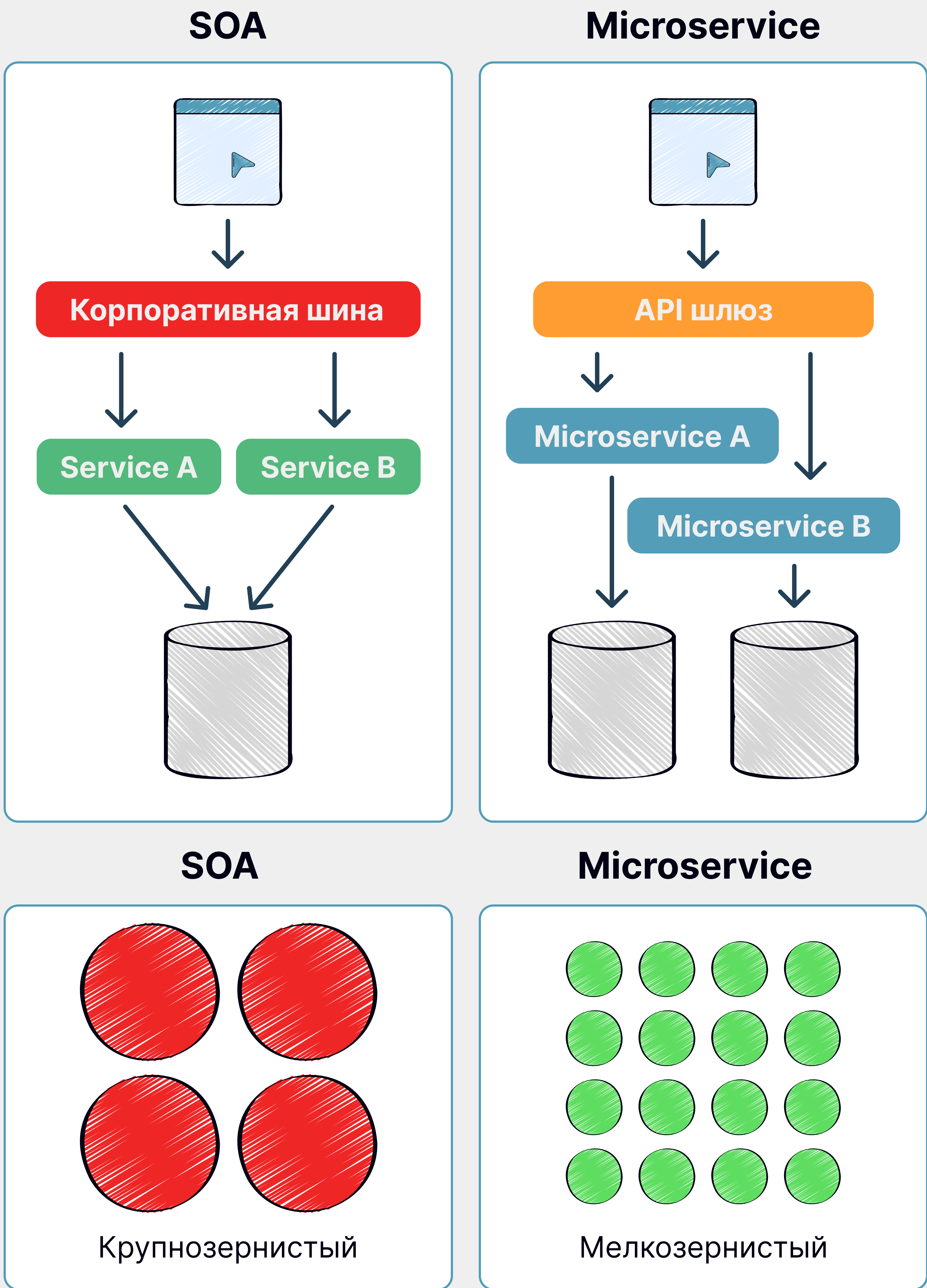


Сравним их различия более подробно с помощью диаграмм.

**Архитектурный стиль SOA** предлагает довольно крупные сервисы, как правило, с централизованным подходом, где сервисы группируются по бизнес-функциям и используются несколькими приложениям

**Стиль микросервисов** предлагает разбиение логики на мелкие сервисы через децентрализованный подход, где небольшие, независимые сервисы выполняют конкретные функции в контексте приложения.

**Методы коммуникации** также эволюционируют со временем. SOA подчеркивает единообразные протоколы связи и стандартизированные интерфейсы для взаимодействия сервисов. Микросервисы склоняются к разнообразным протоколам связи и интерфейсам, часто основанным на REST или очередях сообщений.



Облачные вычисления эволюционировали от инфраструктуры как услуги (IaaS) к платформе как услуге (PaaS) и далее к платформе на основе контейнеров. Таким образом, приложения на основе микросервисов развертываются в контейнерах по умолчанию.

По мере изменения технической архитектуры организационная структура отражает это (закон Конвея). Поэтому с микросервисами структура команды требует многофункциональных продуктовых команд. Каждая команда фокусируется на конкретной области.

|                          |                       |                            |
|--------------------------|-----------------------|----------------------------|
| <b>Развертывание</b>     | Чистое железо IaaS    | Контейнеры                 |
| <b>Методология</b>       | Каскадная Agile       | DevOps                     |
| <b>Структура команды</b> | Изолированная команда | Децентрализованные команды |

## 3. Какие проблемы решают микросервисы?

Архитектура микросервисов появилась в 2010-х годах для решения ключевых проблем:

- **Новая архитектура для облачных вычислений.**

Традиционные программные архитектуры не полностью используют гибкость, которую предоставляют облачные вычисления. Технология контейнеризации (Docker) разбивает ресурсы компьютеров на более мелкие единицы. Инструменты оркестрации (k8s) позволяют легко масштабировать сервисы. Таким образом, архитектура микросервисов возникла для лучшего использования этих достижений облачных вычислений.

- **Разделение сервисов и команд для ускорения работы.**

Монолитные и SOA-приложения часто имеют тесные зависимости между модулями, что затрудняет адаптацию к быстро меняющимся бизнес-требованиям в условиях взрывного роста интернет-индустрии. Архитектура микросервисов разделяет каждый бизнес-домен на ограниченные контексты. Команды продуктов могут сосредоточиться на конкретных доменных требованиях. Сервисы и команды взаимодействуют через заранее определенные спецификации API, что способствует быстрой доставке новых функций и более итеративному тестированию на рынке.

- **Обеспечение надежных методологий разработки.**

Неисправность монолитного приложения может привести к его полному отключению, в то время как микросервисы спроектированы для отказоустойчивости. Неисправности в одном микросервисе изолированы от других сервисов. Дополнительно, инструменты управления предоставляют гибкую распределение запросов или понижение уровня сервиса. DevOps, набор практик и инструментов для улучшения взаимодействия между функциями, естественно вписывается в эту концепцию. Эти изменения методологии позволяют быстро и надежно доставлять продукт.

На [Java Буткемпе](#) мы наглядно показываем участникам, как микросервисы решают проблемы масштабирования и обновления больших приложений. Мы развиваем и разворачиваем отдельные части приложения независимо друг от друга, уменьшая риски и повышая скорость разработки. Это особенно важно для динамичных проектов, где изменения происходят часто.

Еще больше о микросервисной архитектуре простыми словами с красочными анимациями расскажет главный преподаватель FAANG School — Влад Мишустин! Поймешь, чем микросервисы лучше традиционного монолита и еще лучше будешь понимать, что такое микросервисы, для чего они нужны, а главное, как с их помощью получить крутой оффер.

[СМОТРЕТЬ ВИДЕО](#)



## 4. Какие проблемы создают микросервисы?

Хотя архитектура микросервисов решает многие проблемы традиционных архитектур и соответствует эволюции облачных вычислений, она также приносит ряд неудобств.

- **Операционные накладные расходы.**

В системе микросервисов легко настроить контейнеры, что часто приводит к увеличению числа запущенных копий сервисов (инстансов). Мониторинг и отладка этих инстансов без надлежащих инструментов могут превратиться в кошмар.

- **Повышенная сложность.**

Архитектура микросервисов поощряет разделение доменов, поэтому управление коммуникациями и отношениями между сервисами может быть сложным. Необходимо использовать сервисы обнаружения (service discovery) и управления конфигурациями (config management) для эффективного управления инстансами сервисов. Что создает сложности для интеграции и тестирования.

- **Организационная трансформация.**

Согласно закону Конвея, архитектура программного обеспечения формирует организационную структуру. Архитектура микросервисов требует соответствующего структурирования команд.

- **Распределенные транзакции.**

Данные становятся более распределенными, так как каждый домен имеет независимую базу данных. В то время как традиционные архитектуры используют транзакции баз данных для обеспечения согласованности данных, микросервисы могут требовать распределенных транзакций для тех же целей.

- **Производственные накладные расходы и сложное управление ошибками.**

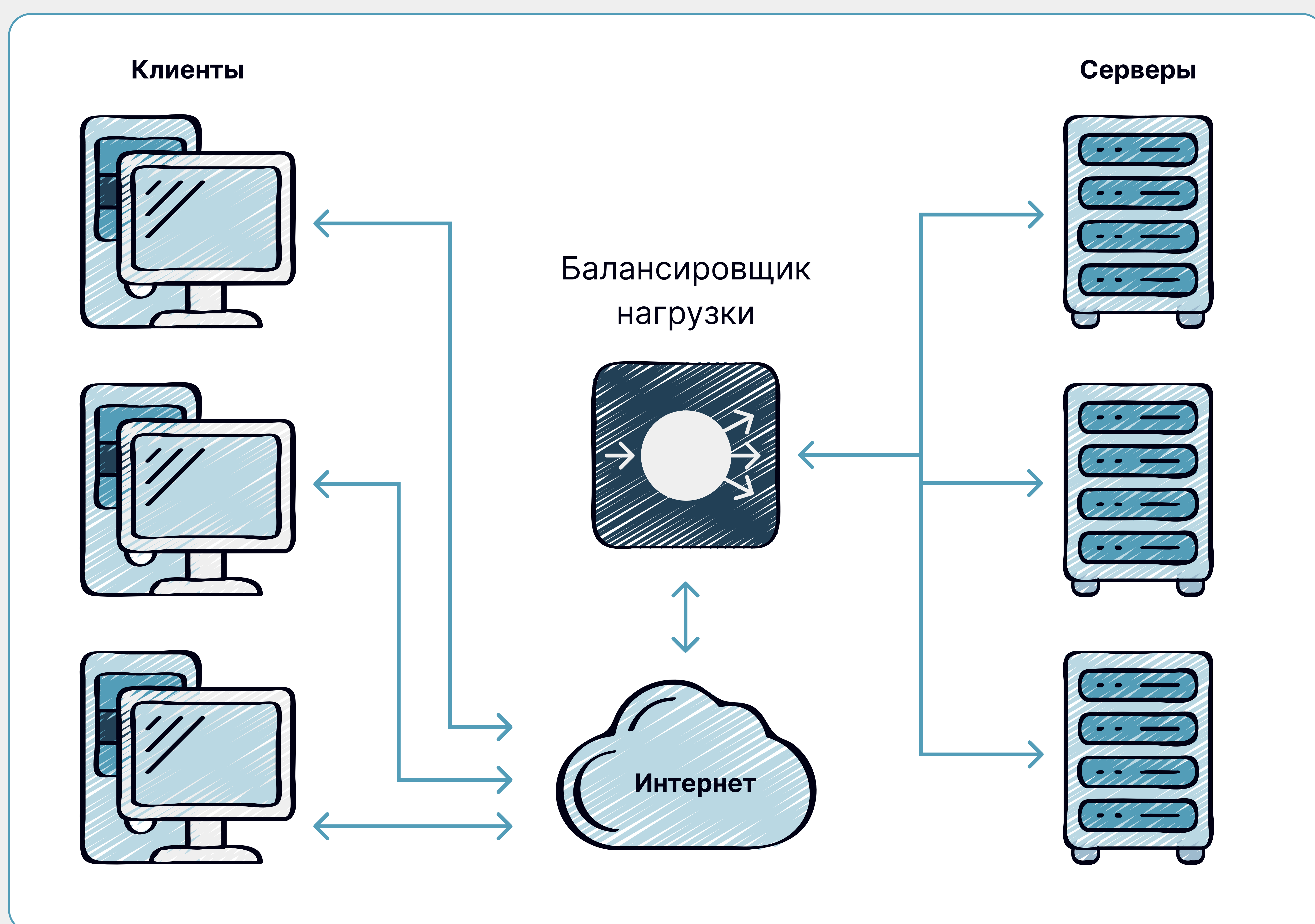
В монолитных приложениях разные модули взаимодействуют через быстрые внутрипроцессные вызовы функций. В случае сбоя функция получает немедленный ответ, что позволяет быстро обработать ошибку. В отличие от этого, микросервисы используют более медленные сетевые взаимодействия, что увеличивает задержку. Также микросервисы обычно взаимодействуют через RPC, что делает обработку ошибок между сервисами менее предсказуемой по сравнению с монолитами.

## 5. Какие основные компоненты архитектуры микросервисов?

Типичная архитектура микросервисов включает:

- **Балансировщик нагрузки.**

Балансировщик нагрузки размещается перед API-шлюзом и распределяет входящие запросы между несколькими инстансами API-шлюза. Это предотвращает возникновение узкого места в одном инстансе шлюза. Общие алгоритмы, используемые балансировщиком нагрузки для распределения запросов, включают round robin, сессии с привязкой и наименьшее количество подключений.

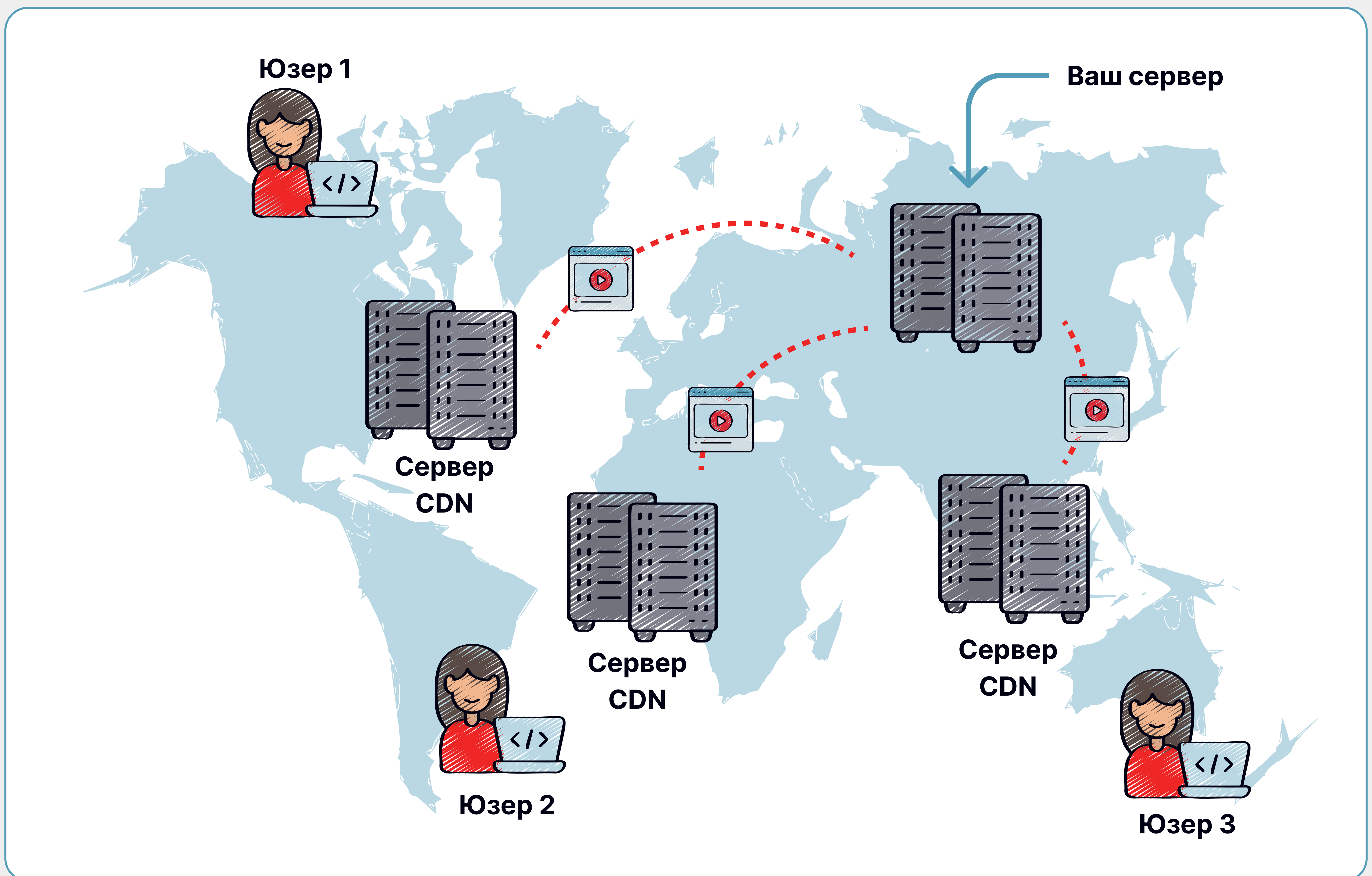


- **API-шлюз.**

API-шлюз обрабатывает все входящие запросы и направляет их к соответствующим микросервисам. Он взаимодействует с провайдером идентификации для обработки аутентификации и авторизации запросов. По сути это именно то место, где происходит авторизация любого запроса к любому микросервису, чтобы не дублировать эту логику в каждом отдельном микросервисе. Шлюз использует компонент обнаружения сервисов (service discovery) для нахождения необходимых микросервисов для конкретных запросов. Кроме того, он может применять правила фильтрации, политики и преобразования запросов перед их направлением на бэкенд.

- **CDN (Сеть доставки контента).**

CDN (Content Delivery Network) представляет собой группу географически распределенных серверов, которые кэшируют статическое содержимое в местах, более близких к конечным пользователям, для ускорения доставки контента. Приложение сначала проверяет CDN для обслуживания статического контента. Если контент не находится в CDN, CDN обрабатывает обратную передачу для получения этого контента с более медленных бэкенд-сервисов.



- **Провайдер идентификации (Identity Provider)**

Провайдер идентификации обрабатывает аутентификацию и авторизацию пользователей.

- **Микросервисы.**

Микросервисы разрабатываются и развертываются вокруг конкретных доменов или областей логики. Каждый домен микросервисов имеет свою базу данных или хранилище данных. API-шлюз взаимодействует с микросервисами, используя REST API или другие протоколы. Микросервисы в пределах одного домена общаются друг с другом через RPC (Удаленный вызов процедур) для внутренних взаимодействий.

- **Регистрация и обнаружение сервисов (Service Discovery)**

Этот компонент управляет регистрацией и обнаружением микросервисов в системе. Чтобы отправить запрос на микросервис, нужно знать, на какой компьютер отправить данные. Поскольку микросервисов много и они запущены в несколько копий на разных компьютерах, все машины регистрируются в одном месте для удобного и быстрого поиска. При запуске новых экземпляров микросервисов они регистрируются в этой службе. API-шлюз использует этот компонент для поиска нужных микросервисов и направления на них запросов.

- **Очередь сообщений (Message Queue)**

Очередь сообщений, как Kafka или RabbitMQ, позволяет микросервисам взаимодействовать асинхронно. Например, служба оплаты может долго обрабатывать транзакции. В этом случае система оформления заказа отправляет запрос в очередь и ожидает ответа асинхронно, избегая задержек в процессах.

- **Кэш.**

Кэш используется для хранения часто запрашиваемых данных, улучшая отзывчивость микросервиса. Однако важно следить за согласованностью данных между кэшем и основными базами данных, чтобы избежать устаревшей информации и неправильного поведения приложения.

- **Хранилище данных: базы данных и объектные хранилища.**

Базы данных и объектные хранилища сохраняют данные приложения. Выбор типа базы данных зависит от модели данных и требований к масштабируемости. Например, в eCommerce-приложении для хранения товаров в корзине может использоваться масштабируемая колонковая база данных, такая как Cassandra.

- **Управление конфигурациями (Config Management)**

Система управления конфигурациями централизует управление и распределение настроек сервисов. Конфигурация должна быть единой для всех копий сервисов, поэтому её нельзя хранить внутри отдельных микросервисов. Вместо этого она хранится в одном месте, доступном для всех микросервисов.

- **Мониторинг и логирование.**

Наблюдаемость через мониторинг и логирование важна для поддержания архитектуры микросервисов. В системах микросервисов много экземпляров сервисов, поэтому автоматизированный мониторинг и распределённая трассировка необходимы для поиска и отладки проблем. Непрерывный мониторинг также помогает проактивно планировать ресурсы и выявлять узкие места.

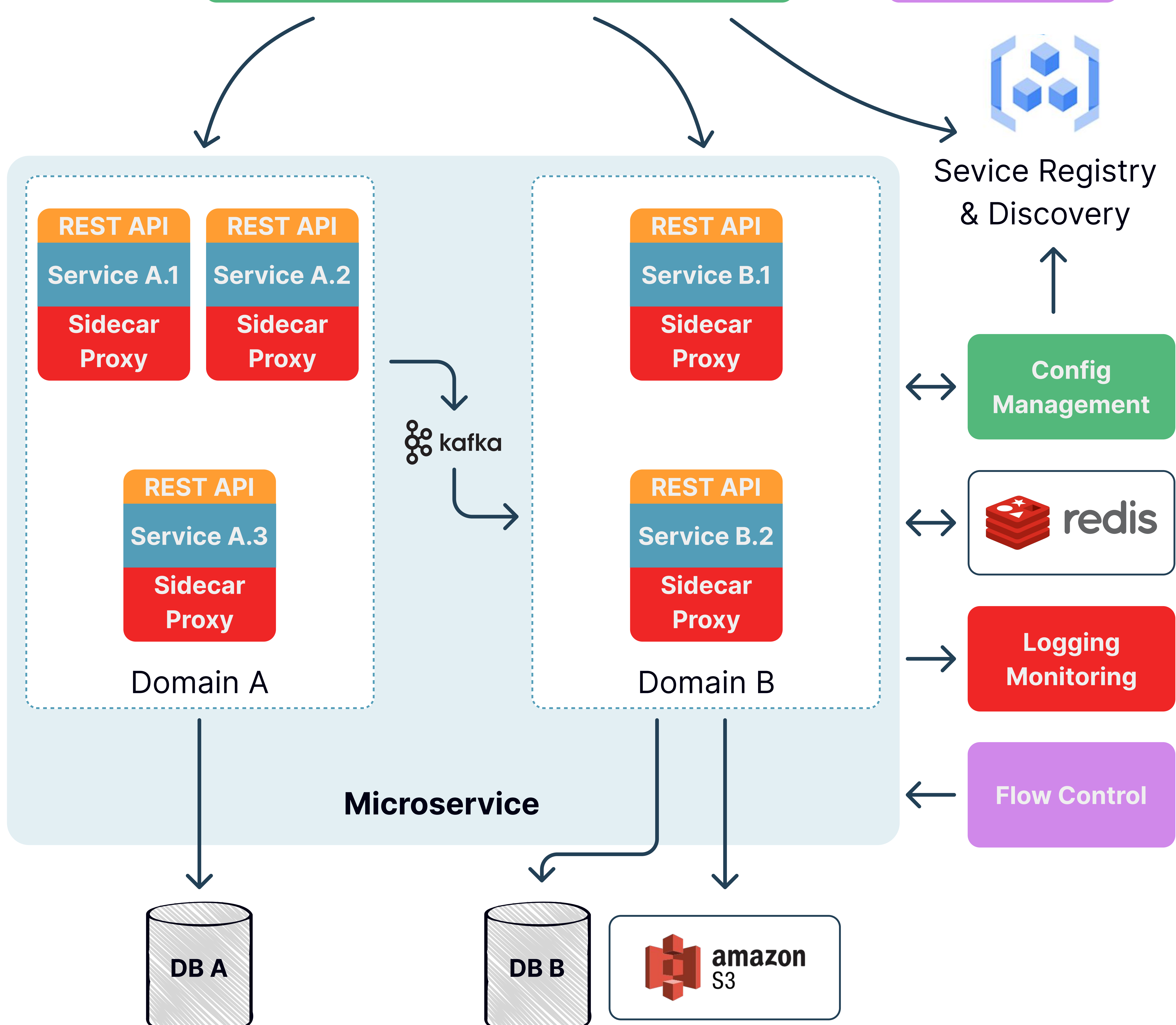
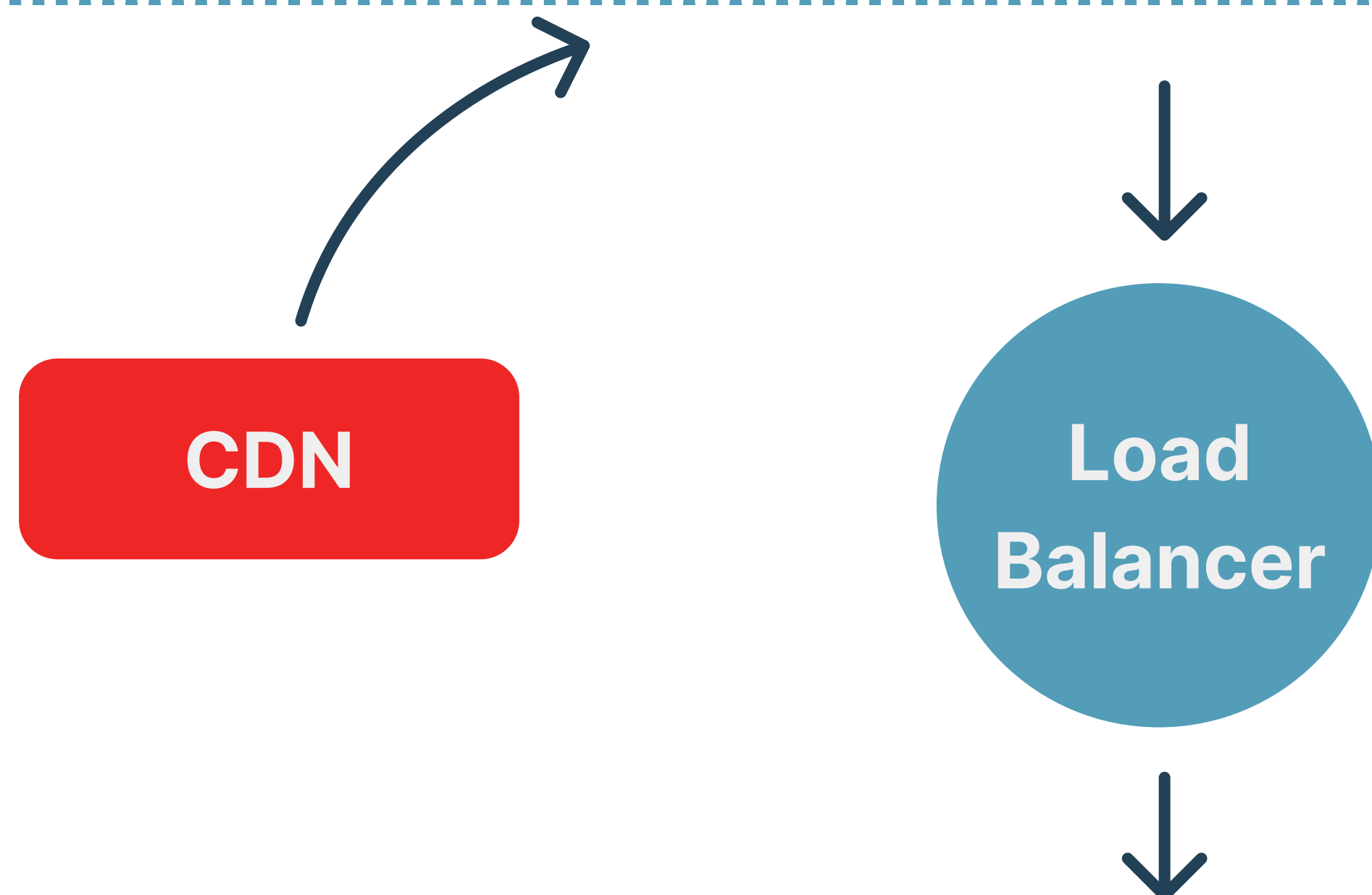
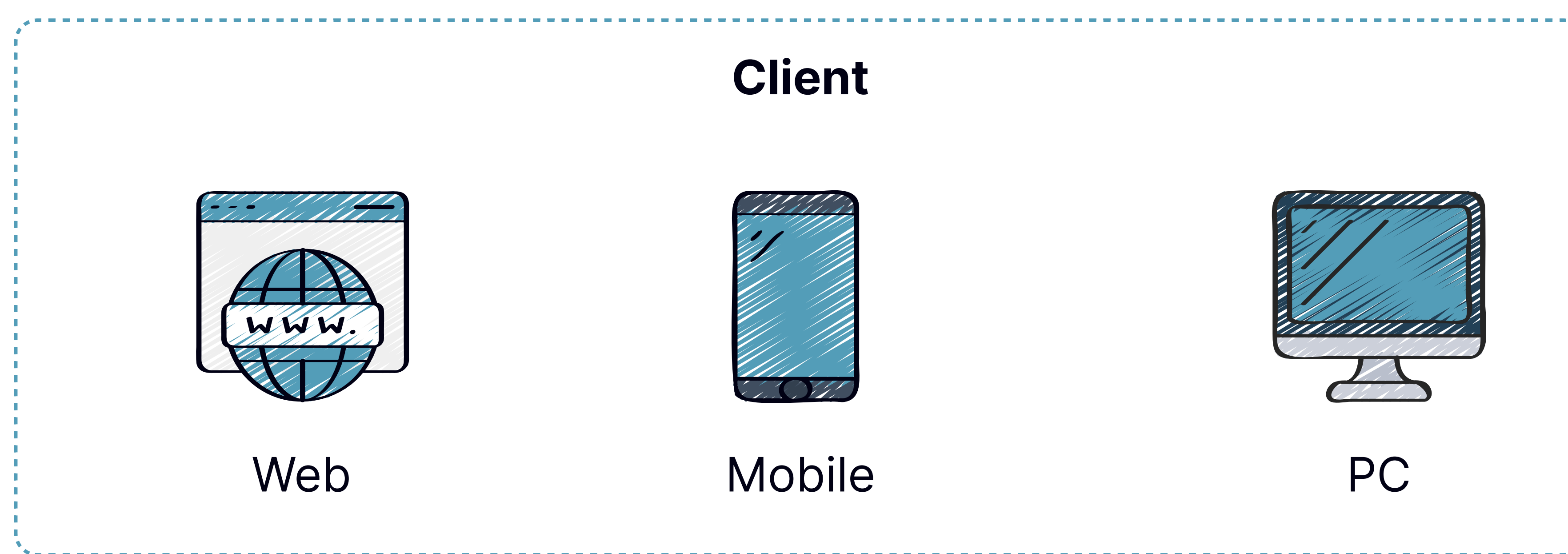
- **Управление трафиком.**

Инструменты управления трафиком необходимы для обработки сбоев в сложных системах микросервисов. Они ограничивают или перенаправляют трафик при проблемах. Например, Hystrix изолирует отказавшие сервисы, а Sentinel управляет потоком трафика, действуя как автоматический выключатель.

- **Сетевые сервисы и оркестрация сервисов.**

Сетевые сервисы предоставляют инфраструктуру для управления коммуникациями и контроля между микросервисами. Они используют прокси-сервер sidecar, развернутый с каждым экземпляром сервиса для управления входящими и исходящими потоками трафика. Это обеспечивает возможности, такие как мониторинг и управление трафиком между сервисами.

Оркестрация сервисов фокусируется на координации последовательности шагов и рабочего процесса. Она использует возможности, предоставляемые сетевыми сервисами, для управления взаимодействиями микросервисов и контроля за выполнением рабочего процесса.



# 6. Какие популярные решения для микросервисов?

### Планирование & Оркестрация



CNCF Graduated



CNCF Incubating



CNCF Incubating



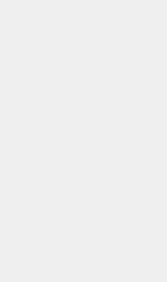
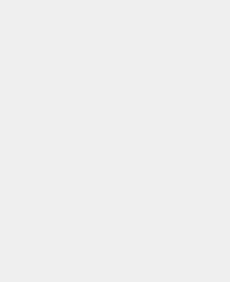
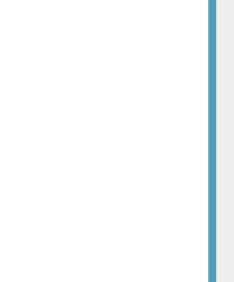
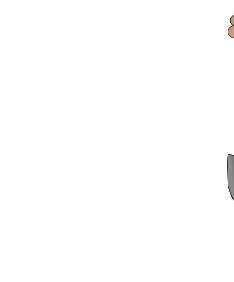
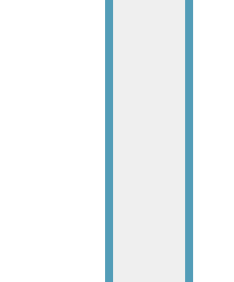
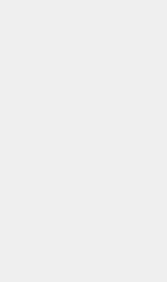
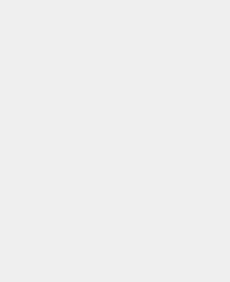
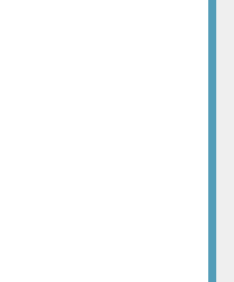
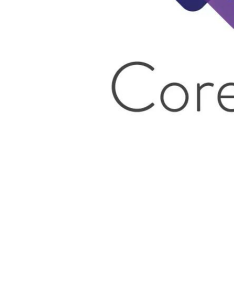
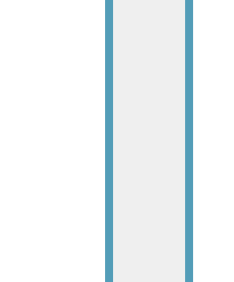
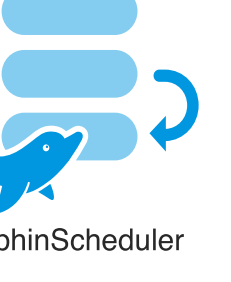
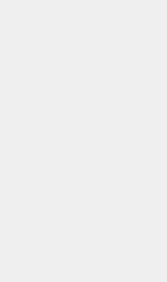
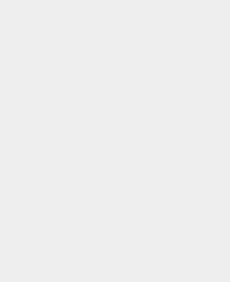
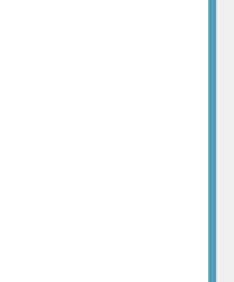
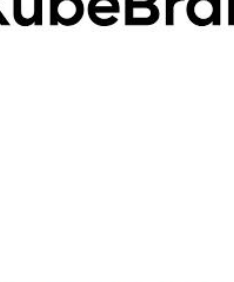
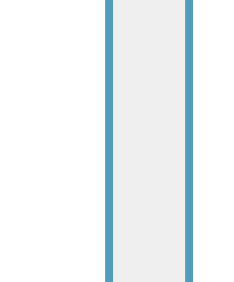
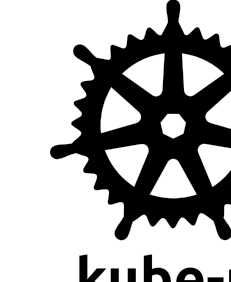
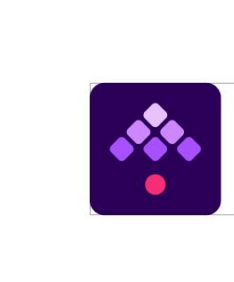
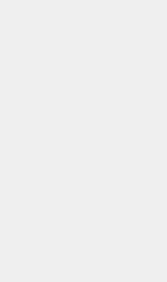
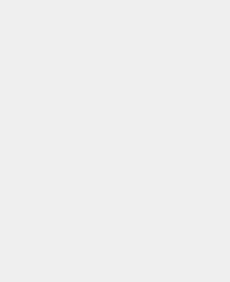
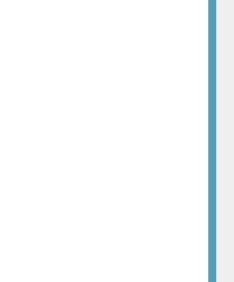
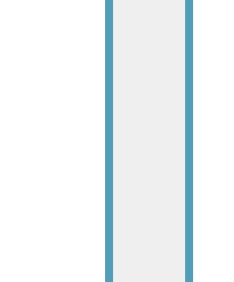
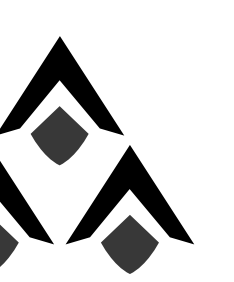
CNCF Graduated



CNCF Incubating



CNCF Incubating



### Координация & Обнаружение сервисов

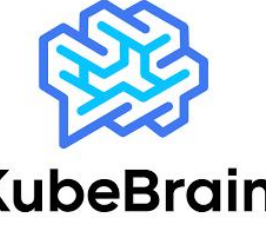


CNCF Graduated

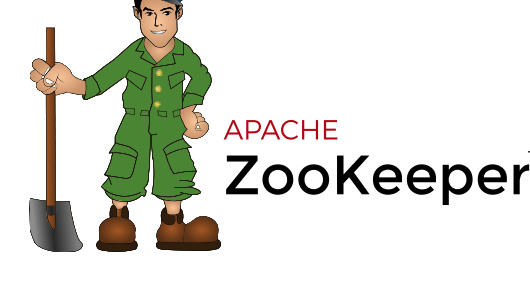


CNCF Graduated









### Прокси сервисы

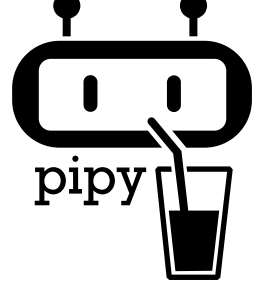


CNCF Graduated



CNCF Incubating









### Удалённый вызов процедур (RPC)

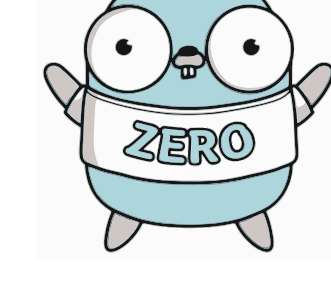


CNCF Incubating









[Spring Cloud](#) — это готовое решение, которое предлагает набор интегрированных инструментов для создания основных компонентов микросервисов.

# 7. Что такое реестр сервисов? (Service Registry)

Реестр сервисов управляет регистрацией и обнаружением сервисов. Он имеет централизованную базу данных для хранения информации о сервисах и их расположении. Реестр направляет запросы клиентов к соответствующим экземплярам микросервисов.

На диаграмме ниже показано, как сервисы регистрируются при запуске и выключении. Существуют два подхода: саморегистрация и регистрация третьей стороной.

При саморегистрации сервисы сообщают о своем статусе в реестр, который затем обновляет таблицу реестра. При регистрации третьей стороной менеджер сервисов управляет жизненными циклами сервисов, запуская и останавливая экземпляры и обновляя реестр. Менеджер сервисов также опрашивает сервисы для мониторинга их состояния.



Давайте рассмотрим еще одну важную функцию реестра сервисов — обнаружение сервисов. На диаграмме ниже показаны два процесса обнаружения сервисов: обнаружение на стороне клиента и обнаружение на стороне сервера.

При обнаружении на стороне клиента, клиенты сначала запрашивают информацию о сервисах в реестре сервисов. Затем они отправляют запросы к API-шлюзу без необходимости искать местоположение. Таким образом, функция обнаружения сервисов выносится за пределы уровня API-шлюза.

При обнаружении на стороне сервера, API-шлюз обрабатывает определение местоположения сервисов. Сначала он получает клиентские запросы и перенаправляет их к соответствующим сервисам. API-шлюз может кэшировать местоположения для ускорения процесса обнаружения. Однако это вводит дополнительную сложность в работу API-шлюза из-за двух проблем:

- 1. API-шлюз должен реализовать логику обнаружения сервисов.
- 2. Поддержание согласованности кэша местоположений с данными реестра является сложной задачей.

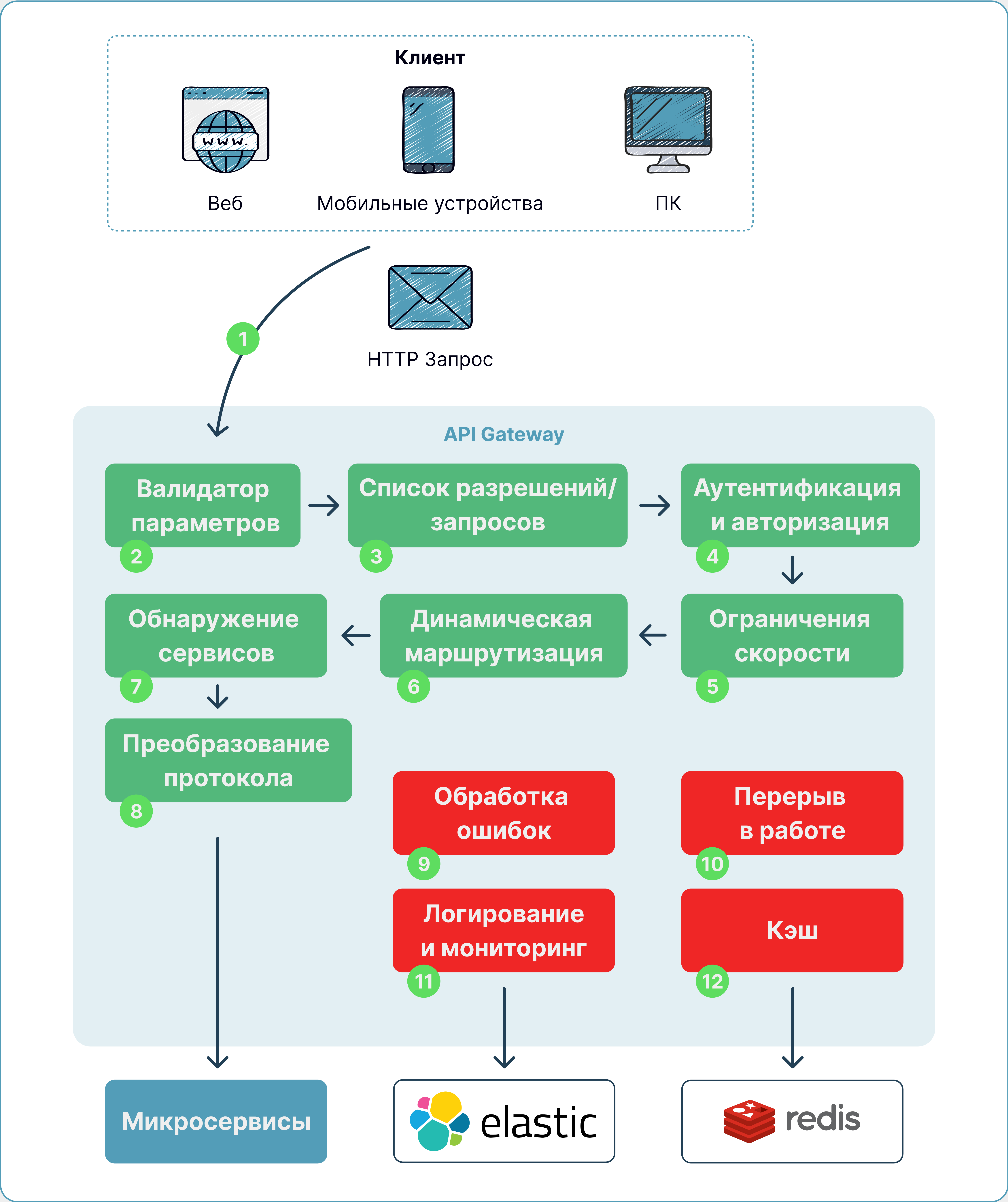


Некоторые примеры продуктов реестра сервисов — это Netflix Eureka и Hashicorp Consul.

# 8. Что такое API-шлюз?

В архитектуре микросервисов API-шлюз выступает в роли единой точки входа для клиентских запросов. API-шлюз отвечает за маршрутизацию запросов, компоновку и преобразование протоколов. Он также предоставляет дополнительные функции, такие как аутентификация, авторизация, кэширование и ограничение скорости.

На диаграмме ниже показаны ключевые шаги:



- Шаг 1: Клиент отправляет HTTP-запрос к API-шлюзу.
- Шаг 2: API-шлюз анализирует и проверяет атрибуты в HTTP-запросе.
- Шаг 3: API-шлюз проверяет списки разрешений и запретов.
- Шаг 4: API-шлюз аутентифицирует и авторизует запрос через провайдера идентификации.
- Шаг 5: Применяются правила ограничения скорости. Запросы, превышающие лимит, отклоняются.
- Шаги 6 и 7: API-шлюз направляет запрос к соответствующему бэкенд-сервису по сопоставлению пути.
- Шаг 8: API-шлюз преобразует запрос в соответствующий протокол и пересылает его к бэкенд-микросервисам.
- Шаг 9: API-шлюз обрабатывает любые ошибки, которые могут возникнуть во время обработки запроса, чтобы обеспечить плавную деградацию сервиса.
- Шаг 10: API-шлюз реализует паттерны устойчивости, такие как предохранители, чтобы обнаруживать сбои и предотвращать перегрузку взаимосвязанных сервисов, избегая каскадных сбоев.
- Шаг 11: API-шлюз использует инструменты наблюдаемости (observability), такие как стек ELK (Elastic-Logstash-Kibana), для логирования, мониторинга, трассировки и отладки.
- Шаг 12: API-шлюз может опционально кэшировать ответы на общие запросы для улучшения отзывчивости.

Кроме маршрутизации запросов, API-шлюз также может агрегировать ответы от микросервисов в один ответ для клиента.

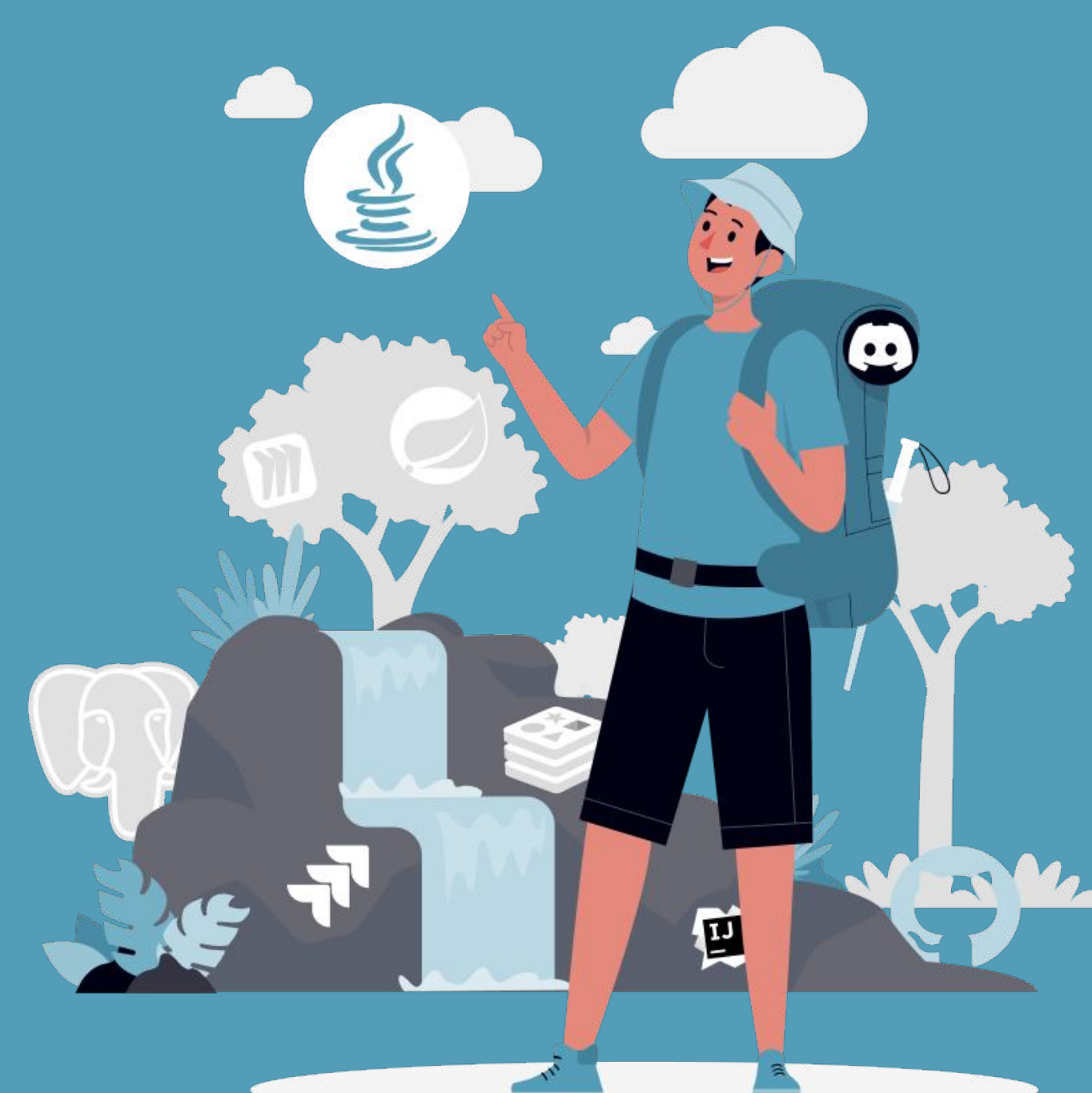
API-шлюз отличается от балансировщика нагрузки. Хотя оба обрабатывают сетевой трафик, API-шлюз работает на уровне приложения, главным образом обрабатывая HTTP-запросы; балансировщик нагрузки в основном работает на транспортном уровне, обрабатывая протоколы TCP/UDP. API-шлюз предлагает больше функций, так как он видит полезную нагрузку запроса.

Однако границы между этими двумя типами инфраструктуры могут стираться.

## Почувствуй себя студентом FAANG School

Запишись на интерактивную экскурсию по Java Буткемпу!

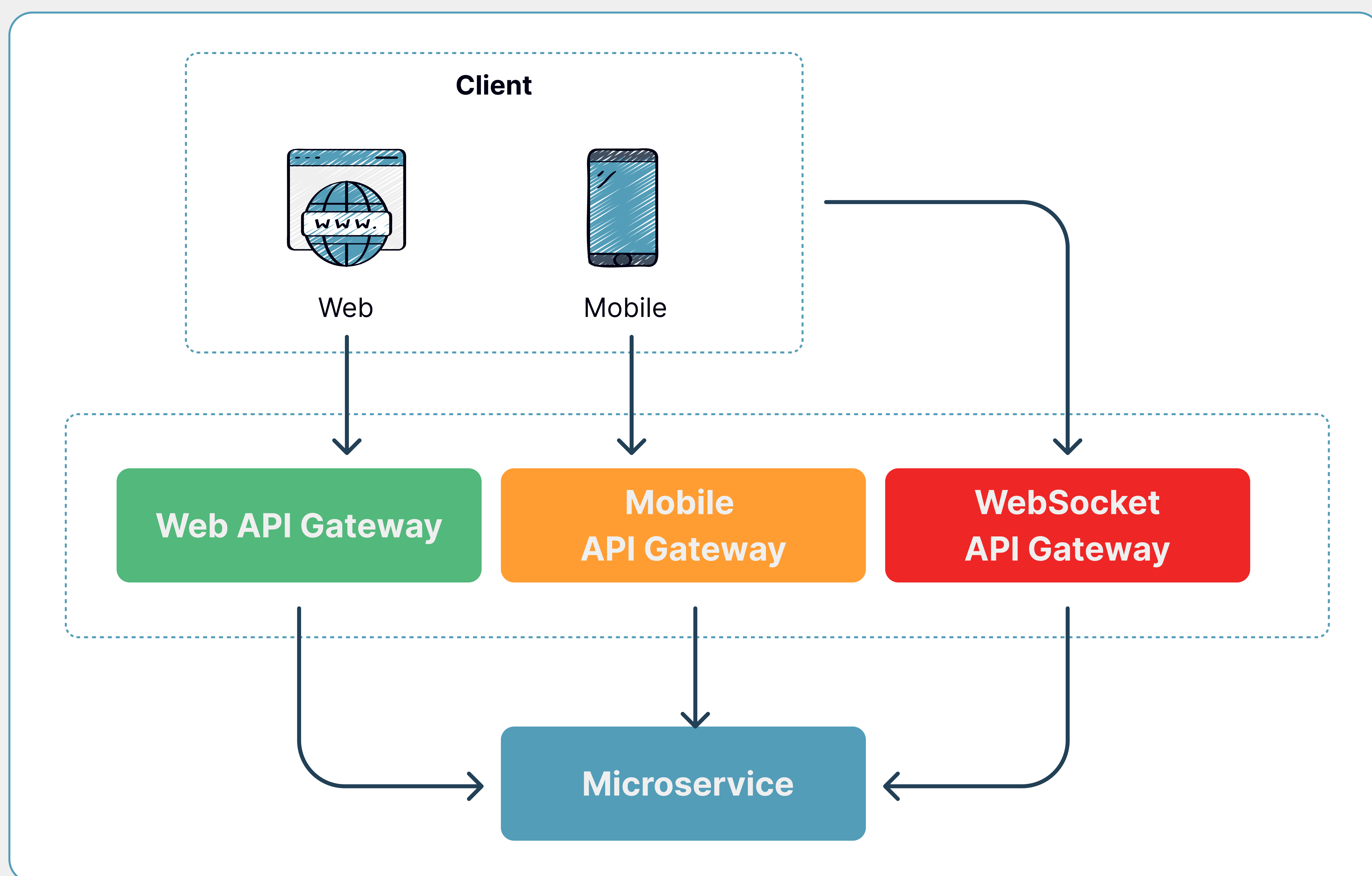
[ЗАПИСАТЬСЯ НА ЭКСКУРСИЮ](#)



Некоторые современные балансировщики нагрузки приобретают возможность видеть и маршрутизировать на уровне приложения, что напоминает функции API-шлюзов.

Но в общем, API-шлюзы фокусируются на проблемах уровня приложения, таких как безопасность, маршрутизация, компоновка и устойчивость на основе полезной нагрузки, в то время как традиционные балансировщики нагрузки сопоставляют запросы с бэкенд-серверами, в основном основываясь на метаданных транспортного уровня, таких как IP и номера портов.

Часто у нас есть отдельные API-шлюзы, адаптированные для различных клиентов и их требований к пользовательскому опыту. На диаграмме ниже показана типичная архитектура. У нас есть разные API-шлюзы для обработки запросов от мобильных устройств и веб-приложений. Кроме того, мы разделяем WebSocket API Gateway, потому что у него другие требования к постоянству соединения и ограничению скорости по сравнению с HTTP-шлюзами.

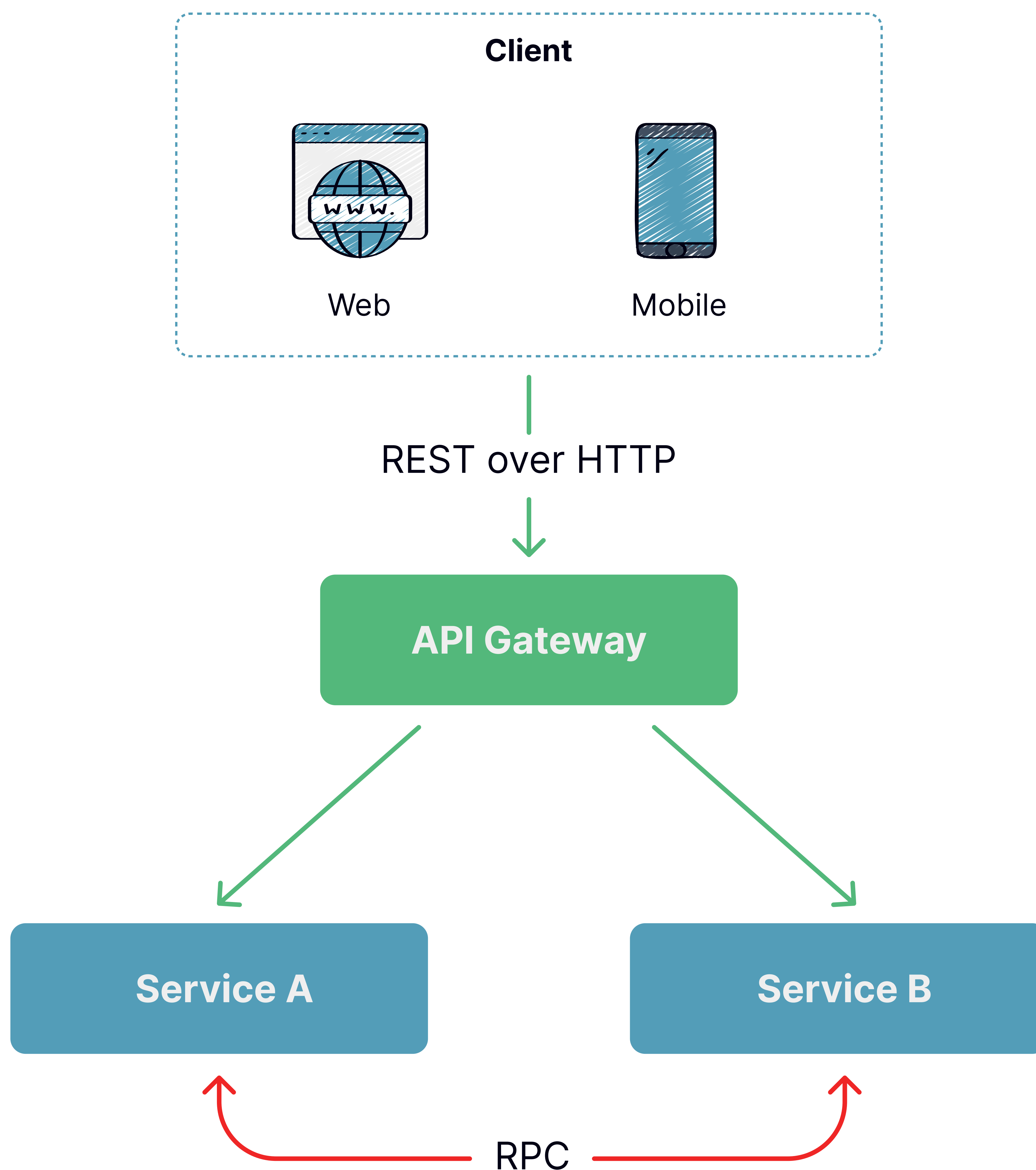


Некоторые последние тенденции API-шлюзов:

1. Поддержка GraphQL. GraphQL — это система типов и язык запросов для API. Многие API-шлюзы теперь предлагают интеграцию с GraphQL.
2. Интеграция с сетями сервисов. Сети сервисов, такие как Istio и Linkerd, используются для обработки коммуникаций между микросервисами. API-шлюзы интегрируются с ними для улучшения управления трафиком.
3. Интеграция с ИИ. API-шлюзы интегрируются с возможностями ИИ для обеспечения более умного маршрутизации запросов или обнаружения аномалий в паттернах трафика.

## 9. В чем различия между REST и RPC?

REST (Representational State Transfer) и RPC (Remote Procedure Call) — это два распространенных архитектурных шаблона, используемых для коммуникации в распределенных системах. REST используется для клиент-серверных коммуникаций, а RPC используется для сервер-серверных коммуникаций, как показано на диаграмме ниже.



### Проверь свои знания в Java!

Пройди тест и узнай, как с твоим уровнем в Java  
стать востребованным программистом.  
Бонус — Roadmap по Java

[ПРОЙТИ ТЕСТ](#)

**QUIZ**

Мы обобщаем их различия в таблице ниже.

- 1. REST основан на стандартных методах HTTP, таких как GET, POST, PUT и DELETE. RPC основан на более быстрых протоколах, таких как HTTP/2.
- 2. REST обычно использует стандартные форматы данных, такие как JSON или XML, для представления ресурсов и их состояний. Форматы данных в RPC могут быть основаны на стандартных JSON или XML, либо на проприетарных форматах, таких как ProtoBuf.
- 3. В REST ресурсы идентифицируются с помощью URI (Uniform Resource Identifiers), тогда как в RPC данные обычно используются для представления команды или действия.
- 4. RPC используется в коммуникациях микросервисов и оптимизирован для более высокой производительности. Например, полезная нагрузка превращается в бинарный формат и является более компактной. REST, с другой стороны, основан на стандартном протоколе HTTP, который используется для клиент-серверных коммуникаций.

|                      | REST                                                                                  | RPC                                                                                                       |
|----------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Протокол связи       | Основан на HTTP глаголах (GET, POST, PUT, DELETE)                                     | gRPC основан на HTTP/2                                                                                    |
| Схема данных         | JSON, XML, HTML, обычный текст                                                        | JSON, XML, ProtoBuf, Thrift, FlatBuffers                                                                  |
| Стиль API            | Ориентирован на ресурсы                                                               | Командно- или ориентирован на действия                                                                    |
| Случаи использования | Публичные API                                                                         | Коммуникации микросервисов                                                                                |
| Производительность   | REST имеет более низкую производительность из-за своего протокола и полезной нагрузки | RPC имеет более высокую производительность благодаря HTTP/2 и компактной закодированной полезной нагрузке |

# 10. Что такое менеджер конфигураций?

В архитектуре микросервисов каждый сервис должен поддерживать множество конфигураций, таких как подключения к базе данных, порты и уровни логирования. Эти настройки централизованно управляются и хранятся в менеджере конфигураций, упрощая управление для сотен сервисов. Настройки часто различаются в зависимости от окружения (тестовые и продакшн сервера).

Менеджер конфигураций выполняет следующие обязанности:

- 1. Централизованное управление настройками:** сервисы получают конфигурации из распределённого хранилища ключ-значение, избегая выпуска новых версий для изменения конфигураций.
- 2. Обновление конфигураций в реальном времени:** позволяет сервисам перезагружать обновления без перезапуска, облегчая непрерывную доставку изменений.
- 3. Версионирование конфигураций:** отслеживает историю изменений, связывает их с изменениями кода и позволяет откатывать конфигурации вместе с кодом.
- 4. Контроль доступа:** правила видимости на основе ролей позволяют командам просматривать или изменять конфигурации своих микросервисов. Например, только команда SRE может просматривать конфигурации в рабочей среде, а команда платежей изменять адреса платежных каналов.
- 5. Конфигурации для разных сред:** различает конфигурации тестовой, QA и промежуточной сред от рабочей среды, используя аналогичные рабочие процессы для развертывания.
- 6. Поддержка многопользовательской среды:** позволяет настраивать разделение конфигураций в многопользовательских сценариях, изолируя их для предотвращения конфликтов между клиентами.

Менеджер конфигураций должен быть высокодоступным. Недоступность хранилища конфигураций может привести к сбоям в работе сервисов и масштабным системным сбоям. Для повышения устойчивости менеджеры конфигураций используют кластеризацию, репликацию и избыточность. Популярные решения с открытым исходным кодом включают HashiCorp Consul, etcd и Apache ZooKeeper, а также облачные решения, такие как Spring Cloud Config Server.

Менеджеры конфигураций интегрируются с инфраструктурой, такой как оркестраторы контейнеров. Например, Kubernetes использует etcd для обнаружения сервисов и координации компонентов, что обеспечивает централизованное управление конфигурациями и управление кластером. Это показывает, как менеджеры конфигураций выполняют ключевые роли в современных средах.

# 11. Какие существуют общие подходы к обеспечению отказоустойчивости микросервисов?

Гарантия отказоустойчивости является ключевым аспектом при разработке архитектур микросервисов. Netflix создал библиотеку под названием Hystrix для обеспечения задержек и отказоустойчивости микросервисов. Она реализует несколько общих паттернов устойчивости, которые остаются актуальными даже с появлением новых инструментов.

Hystrix внедрил паттерны, такие как предохранители и резервные варианты, которые полезны для создания надежных микросервисов. Хотя Netflix теперь рекомендует Resilience4j, как более современную библиотеку устойчивости, основные подходы, используемые в Hystrix, по-прежнему применимы. Давай рассмотрим некоторые из этих ключевых техник:

- **Предохранитель (Circuit Breaker).**

Паттерн предохранителя отслеживает запросы к сервисам. Если уровень отказов превышает порог, «срабатывает» предохранитель, чтобы остановить отправку запросов, которые, вероятно, завершатся неудачей. Это предотвращает каскадные сбои между взаимосвязанными сервисами.

- **Резервный вариант (Fallback).**

Резервный вариант предоставляет ответ по умолчанию или пониженной функциональности, если вызов сервиса завершается неудачей или истекает время ожидания. Это обеспечивает более плавный пользовательский опыт во время сбоев.

- **Кэширование запросов.**

Кэширование результатов повторяющихся запросов уменьшает количество дублирующих вызовов к основным сервисам для получения одних и тех же данных.

- **Объединение запросов.**

Объединяет несколько одновременных запросов на одни и те же данные в пакеты. Это позволяет избежать дублирующих одновременных вызовов на один и тот же ресурс. Вместо этого запросы группируются и отправляются как единый агрегированный пакет. Это снижает нагрузку на downstream-сервисы.

- **Изоляция отсеков.**

Изоляция отсеков устанавливает лимиты использования ресурсов для предотвращения каскадных отказов между компонентами и сервисами. Например, пулы потоков могут быть настроены с максимальными пределами одновременного выполнения для создания границ изоляции. Если один компонент или сервис начинает выходить из строя или потреблять слишком много ресурсов, отсеки обеспечивают «сдерживание» отказов и истощения ресурсов в этих границах, вместо того чтобы распространяться по всей системе. Это предотвращает полные сбои системы.

- **Мониторинг состояния.**

Отслеживание метрик состояния позволяет принимать решения о порогах срабатывания предохранителей, размерах отсеков и других параметрах устойчивости.

# 12. Как мы управляем распределенными транзакциями?

**Транзакция** — это единица работы, состоящая из последовательности операций, выполняемых как одно, неделимое и атомарное целое в системе. Транзакции обладают свойствами ACID:

- **Атомарность** — Последовательность операций выполняется полностью или не выполняется вообще.
- **Согласованность** — База данных остается в согласованном состоянии после завершения транзакции.
- **Изоляция** — Конкурирующие транзакции изолированы друг от друга.
- **Долговечность** — Данные сохраняются после завершения транзакции, даже в случае сбоев системы.

Транзакции становятся «распределенными» из-за того, что базы данных становятся распределенными для обработки больших объемов данных. Распределенные транзакции стремятся следовать свойствам ACID, но их достижение вводит дополнительные сложности по сравнению с транзакциями в централизованной базе данных. Давай рассмотрим свойства:

## 1. Атомарность.

Распределенная транзакция может охватывать несколько ресурсов. Банально нужно выполнить разные запросы в разных базах данных. Если любая часть выходит из строя, все ресурсы должны быть откатаны назад, что сложно координировать. В этом сценарии можно использовать более упрощенный протокол, такой как двухфазный коммит (2PC).

## 2. Согласованность.

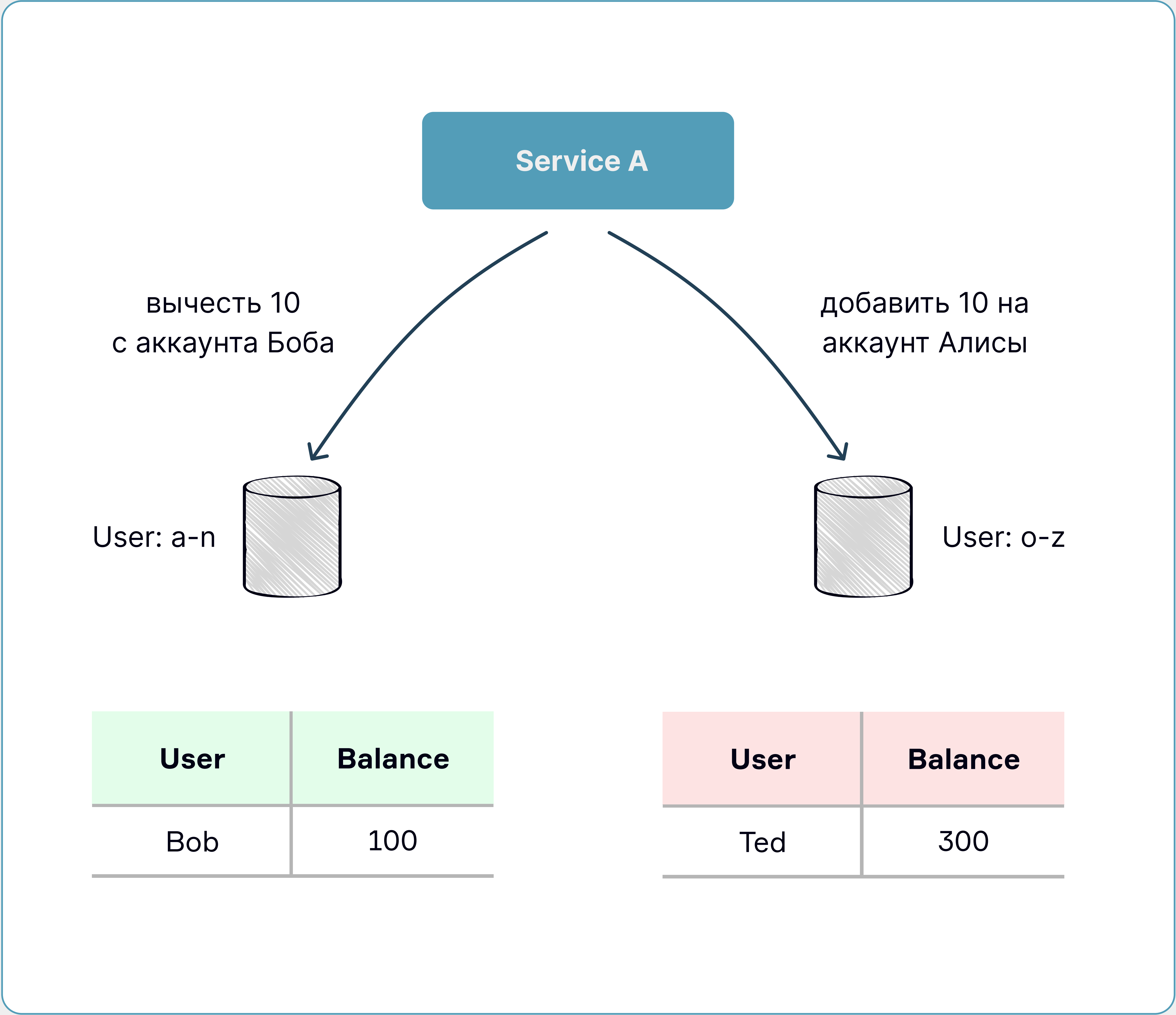
В распределенной среде это свойство означает, что несколько баз данных должны поддерживать согласованное состояние друг с другом. Если одна база выходит из строя или теряет данные, общее состояние системы становится несогласованным. Чтобы справиться с этим, мы часто применяем «eventual consistency» в качестве компромисса. Идея заключается в том, что даже если некоторые данные временно несогласованы между нодами, со временем ноды «в конечном итоге» вернутся к согласованному состоянию. Таким образом, мы ослабляем абсолютную гарантию согласованности в пользу того, чтобы система в целом достигала согласованности со временем, принимая короткие периоды несогласованности для улучшения доступности и устойчивости к разделению. Нода (node, узел) — один запущенный сервер с БД/микросервисом на нем. То же самое, что instance.

### 3. Изоляция

Конкурирующие транзакции между нодами могут вызывать конфликты ресурсов. Это сложно в распределенных средах. Оптимистическое блокирование (optimistic locking) или изоляция снимков могут помочь достичь изоляции распределенных транзакций.

### 4. Долговечность

Данные должны сохраняться на достаточном количестве распределенных нод, чтобы восстановить данные, даже если некоторые ноды выйдут из строя. Это связано с согласованностью - сохраняемые данные также должны оставаться согласованными. Обычно данные реплицируются (копируются) на нескольких нодах для обеспечения избыточности (redundancy). Алгоритмы консенсуса, такие как Raft или Paxos, могут устанавливать согласие между узлами по поводу правильной "золотой копии" - самого правильного состояния данных, к которому обратится кто-то другой.



Управление распределенными транзакциями — сложная задача. Некоторые фреймворки и паттерны помогают справиться с этой задачей:

- **Try-Confirm-Cancel (TCC).**

TCC — это паттерн распределенных транзакций. Он делит каждую транзакцию на этапы Try, Confirm и Cancel. Этап Try пытается выполнить свою локальную транзакцию. Этап Confirm фиксирует её. Этап Cancel откатывает транзакцию, если Try или Confirm завершились неудачно. TCC обеспечивает согласованность без использования центрального координатора.

- **Двухфазный коммит (2PC).**

2PC — это протокол, который обеспечивает атомарность и согласованность распределенных транзакций. Все участники либо фиксируют свои изменения, либо откатываются к исходному состоянию. Он состоит из двух фаз — подготовки и фиксации. На этапе подготовки централизованный координатор (менеджер транзакций) отправляет запрос каждому участнику с просьбой подготовиться к транзакции. Участники отвечают либо голосом за фиксацию, либо за отмену. На этапе фиксации, если все участники проголосовали за фиксацию, координатор отправляет сообщение "commit", инструктируя их сделать изменения постоянными. Если кто-либо проголосовал за отмену или произошла ошибка, координатор отправляет сообщение "rollback", откатывая все ноды к их состоянию до транзакции на этапе подготовки.

- **Saga.**

Паттерн Сага управляет распределенными транзакциями без центрального координатора. Вместо этого он делит длительную транзакцию на последовательность более мелких, независимых транзакций (саг). Каждая сага управляет своей локальной логикой фиксации и отката. Это обеспечивает более гибкое и масштабируемое управление распределенными транзакциями. Если сага завершается неудачей, она компенсирует свои изменения, запуская компенсирующую транзакцию, которая нивелирует предыдущие изменения, сохраняя согласованность системы.

**Например, перевод \$10 от Боба к Алисе может включать такие саги:**

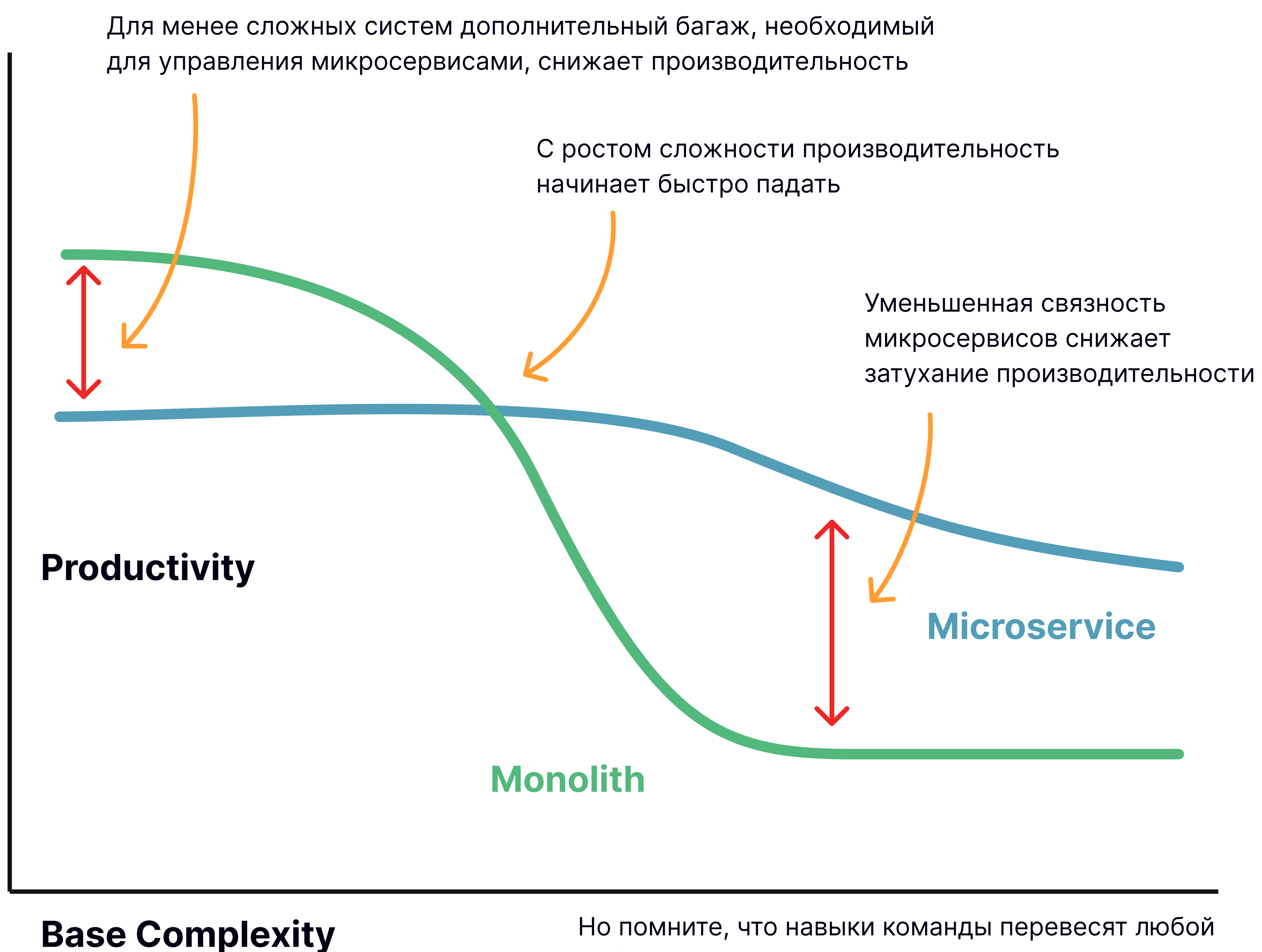
- Снять \$10 с аккаунта Боба - Компенсировать, вернув \$10 обратно на аккаунт Боба
- Положить \$10 на аккаунт Алисы - Компенсировать, сняв \$10 с аккаунта Алисы

Каждая сага управляет своими границами транзакции. Если депозит не удался, сага кредита компенсируется снятием ошибочного депозита. Это сохраняет согласованность системы. Ключевое преимущество паттерна Сага — децентрализация: не нужен центральный координатор, что позволяет гибкое и масштабируемое управление распределенными транзакциями. Участники должны координировать свои действия только с предшественником и последователем своей саги, а не с глобальным координатором. Компромисс состоит в том, что саги жертвуют атомарностью ради доступности. Изменения становятся в конечном итоге согласованными, а не идеально скоординированными.

## 13. Как выбрать между монолитной и микросервисной архитектурами?

Выбор между монолитной архитектурой и микросервисами не всегда прост. Существуют преимущества и недостатки каждого подхода, которые следует тщательно взвесить в зависимости от конкретных потребностей и контекста программной системы и команды разработки.

Как показано на диаграмме Мартина Фаулера, мы обычно рекомендуем начинать с монолитной архитектуры, когда бизнес-потребности просты, программная система еще не сложна, а команда разработки относительно невелика. По мере увеличения сложности со временем монолитные системы становятся трудными для поддержки и масштабирования. В этот переломный момент переход к микросервисной архитектуре позволяет добиться лучшей модульности, масштабируемости и гибкости технологий. Подход микросервисов также позволяет быстрее вводить новые функции для больших инженерных команд, работающих в нескольких доменах.



На [Java Буткемпе](#) мы подробно изучаем и анализируем, когда стоит выбрать монолитную архитектуру, а когда — микросервисную. Монолиты хороши для простых и небольших приложений, где важна скорость разработки. Микросервисы, напротив, подходят для сложных систем, которые требуют масштабирования и частого обновления отдельных компонентов без нарушения работы всего приложения.

Надеемся, что этот гайд был полезен и ты будешь применять полученные знания на практике!

Если **остались вопросы**, команда FAANG School с радостью ответит на них, для этого переходи [по ссылке](#) и оставляй заявку! Расскажем, как именно тебе бустануть карьеру в IT и дадим конкретные рекомендации.

**Подписывайся  
на нас в соц.сетях:**

 [Telegram](#)

 [YouTube](#)

 [VK](#)

